

# 4

## Advanced C Programming: Structs, Pointers, and Arrays + Introduction to Assembly

### Objectives:

Some of the advanced elements of C include structs, pointers, and arrays. When you complete this lab, you should be able to:

- Create C structs, pointers, and arrays.
- Understand the fundamentals of assembly programming.
- Recognize and use a few basic Thumb-2 instructions and their corresponding assembly mnemonics.

### Related material to read:

- C Text, Chapters 3-5 in General
- Text: Embedded Systems with ARM Cortex-M3 Microcontrollers in Assembly Language and C, Chapter 3 (ARM Instruction Set Architecture) (*Section 3.3 Optional*)
- Appendix A (ASCII Table)

## 4.1 Introduction

This will be a one-week lab. Lab reports will be due a week from today. This lab will go over advanced C programming concepts like structs, pointers, and arrays. We will also introduce basics of Assembly.

## 4.2 C Pointers

C pointers allow you to create variables that “point” to a memory location. In most cases, we will use these C pointers to point to another variable (remember that a variable is always held in a memory location). Pointers can be useful and make certain tasks easier. The value of a pointer is always the memory address to which the pointer is pointing to. We use the following syntax to declare a pointer:

```
datatype * ptrVar;  
int *ptrInt;
```

to set a pointer to point to a particular variable, we use the & operator

```
int testInt;  
ptrInt = &testInt;
```

and to access that variable from the pointer, we use the \* operator

```
*ptrInt
```

A pointer in C is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -. To understand pointer arithmetic, let us consider the ptrInt in the previous examples. This pointer points to the address that contains testInt, let us assume that is address 1000 in this example. When we increment or add one to the pointer (++ptrInt), ptrInt will actually point to 1004. This is because pointer increment moves the pointer to the next *integer* and since integers are four bytes, it is modified to 1004. The key thing to note is that pointer increment adjusts the pointer based on the size of the datatype it is pointing to. Similar behavior with ptrInt + 2, it refers to the integer that is two integer sizes (8 bytes) away in memory.

**(10 pts) QUESTION A: If double \*ptrChar is pointing to address 0x2010 and we perform ++ptrChar, what is the new address that ptrChar is pointing to?**

## 4.3 C Arrays

In C, in addition to the basic built in types like char, int, or float, we can create grouping of related data items called arrays. These arrays are used to group up data items of the same type. In order to declare an array, we use the following syntax:

```
dataType name [numElements]  
int testArray[10];
```

where `testArray` is a series of 10 integers. This is nifty to keep a single variable for a series of values. For example, an array to hold a series of grades for the class, or an array to hold a series of characters that represent a person's name. To access a particular element of the array, we use the following syntax:

```
int index = 0;
testArray[index];
```

The above code accesses the 0-index element or the first element in the array (indices start from 0 in C). If I want to initialize the array to some predetermined value, I can initialize them using curly braces:

```
int testArrayTwo[5] = {1,4,3,2,0};
testArrayTwo[0] //Has a value of 1
testArrayTwo[1] //Has a value of 4
testArrayTwo[2] //Has a value of 3
testArrayTwo[3] //Has a value of 2
testArrayTwo[4] //Has a value of 0
```

Please note that you can only do this at array declaration. You can't assign the array to these values later on.

### (10 pts) QUESTION B: What is the maximum index we can use for `testArray`?

**Special Case – Strings:** In C programming, we have a special class of arrays called strings. They are char arrays that have a bunch of (typically) ASCII characters that end in the null character `'\0'` or value 0. The string literals that everyone has been using, e.g., `"Hello, Lab4!\n"` follow this specification. Example:

```
"Hello" = ['H', 'e', 'l', 'l', 'o', '\0']
```

## 4.4 C Structs

Another type of grouping of related data items are called `struct`. These structs are especially useful if we want to group together data items of different types. For example, for a student record, we could group up a string array for their name, an integer for the id number, an array that holds their course history, and an integer for the number of courses taken. That is:

```
struct student
{
    char firstName[20];
    int id;
    struct course courseHistory[20];
    int numCoursesTaken;
};
```

The above creates a struct called `student` (the new type is `struct student`). That has a char array of size 20 for their first name, an int for id, an array of a different struct called `course` (holds

information for each course), and an int for the number of courses taken. Then to use this struct, like any other type, I can create a variable called test of type struct student:

```
struct student test;
```

Then to access any of the values inside the struct, we use the “.” operator.

```
test.id = 1234567; //sets the student's id to 1234567
test.courseHistory[2]; //this accesses the student's third
course
```

Also like any other type, we can create pointers that point to struct student variables.

```
struct student * testptr = &test;
```

Since dereferencing these pointers can get a little messy (e.g., (\*testptr).id) C introduces a new operator to access values inside of the struct from a pointer, the “->” operator.

```
testptr->id = 1234567; //since testptr points to test; this
does the same thing as test.id = 1234567
testptr->courseHistory[2]; // since testptr points to test;
this does the same thing as test.courseHistory[2]
```

Lastly, although we have defined the struct student, we always have to write struct student whenever we are using that struct. This can be cumbersome, thankfully, we can officially define a new type using the typedef keyword.

```
typedef type newType;
typedef struct student Student;
```

We can do this to create new type names from any other type definition.

#### 4.5 Problem 4a: GPA Calculator

In the first problem of this lab, we will adjust the code for problem 3c in Lab 3, the GPA calculator. Instead of just keeping track of the classes, we will create a struct that contains the student's records. We will assume that the student can get from 'A' to 'F' and no '+' or '-' grades.

##### Grades to Grade Points

- A = 4 points
- B = 3 points
- C = 2 points
- D = 1 points
- F = 0 points

##### **Procedure**

To start, create a Platform.IO project using the same procedures as Lab 1 and using the files in the “Problem4a” folder in the provided zip file. The only difference is that you should use main4a.c

instead of main.c and include both student.c and student.h. Define functions where appropriate! Comment all functions. Similarly, modify the comments at the top of main4a.c and student.c to match your details.

1. Create a struct called course with the following member variables (I have already given student as an example):
  - a. Character array of size 30 that will contain the course's name
  - b. Integer that will contain the course's id (Normally would be the CRN)
  - c. Character that contains the grade the student got in this course
  - d. Integer that contains the number of credits for this course
2. Typedef this course struct so you can use "Course" instead (I have already given Student as an example)
3. Create the following functions in student.c and prototypes in student.h (**Follow the function prototypes, for these functions only use standard C constructs and UARTprintf. No other external functions allowed.**)
  - a. createStudent
    - i. Description: Creates a student variable. Sets the name given the input. Sets the id given the input. Set the number of courses taken to 0 (we haven't added any yet).
    - ii. Considerations: Your student's firstName should end in '\0'. Do not exceed the boundaries of firstName (20 characters).
  - b. addCourse
    - i. Description: Modifies student to add a course using the input parameters and modify the number of courses taken by student.
    - ii. Considerations: You should not exceed size of the Course array.
  - c. printStudent
    - i. Description: Prints out the student information including courses taken.
    - ii. Considerations: If you assigned everything using C string notation, you can use UARTprintf("%s", var);
  - d. calcGPA
    - i. Description: calculates the GPA for the student given their classes.
4. Call the functions to create a student and add some classes. Test to make sure this all works properly. (**you don't need to get user input just call the functions with some predetermined values**)

Call the lab TA and demonstrate the program working. The TA will enter their own test inputs, if your program outputs the correct values the lab TA will check you off. The TA will also scan through your functions to see if they were done correctly.

5. Create an array of three students, and repeat step 4 for each student.

Call the lab TA and demonstrate the program working. The TA will enter their own test inputs, if your program outputs the correct values the lab TA will check you off. The TA will also scan through your functions to see if they were done correctly.

Example Output:

```

--- Quit: Ctrl+C | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
Hello, Lab 4!
Problem 4a

-----
Student name: Bob
Student ID: 1234
  Class: Intro to Basketweaving CRN: 101      Grade: A      Credits: 2
  Class: Relativity and Quantum Mechan CRN: 500      Grade: C      Credits: 3
  Class: Intro to Microcontrollers CRN: 251      Grade: B      Credits: 3
GPA: 2.875000

-----
Student name: Bob
Student ID: 1234
  Class: Intro to Basketweaving CRN: 101      Grade: A      Credits: 2
  Class: Relativity and Quantum Mechan CRN: 500      Grade: C      Credits: 3
  Class: Intro to Microcontrollers CRN: 251      Grade: B      Credits: 3
GPA: 2.875000

-----
Student name: Fred
Student ID: 2345
(No classes)
GPA: 0.000000

-----
Student name: Ann
Student ID: 5678
  Class: Relativity and Quantum Mechan CRN: 500      Grade: A      Credits: 3
  Class: Intro to Microcontrollers CRN: 251      Grade: A      Credits: 3
  Class: Suffering Fools Gladly CRN: 102      Grade: D      Credits: 2
  Class: Honors English CRN: 300      Grade: A      Credits: 3
GPA: 3.454545

```

## 4.6 Assembly Programming Fundamentals

An assembly language, like many other computer languages, consists of a series of statements that tell the CPU what operations to perform. An assembly language is composed of four types of statements: *directives*, *instructions*, *comments*, and *labels*. Assembler **directives** provide

valuable information for assisting the assembler. For example, these assembler directives can be used to define program constants and reserve space for program variables. Assembly **instructions** are divided into two fields: Operation (or Mnemonic) and Operand fields. The Operation field consists of the mnemonic names for the machine instructions. The Operand field contains the operand(s) and assembler directive arguments (if necessary). The **comment** field is an optional field that allows programmers to document their code. Comment fields are ignored by the assembler and begin with a double slash '//'. Finally, there is another optional field for **labels**. A label allows a programmer to use symbols for memory locations, which make assembly programs easier to read and write. As it turns assembly code into object code, the assembler “replaces” each label with its respective memory location. Labels are used to easily refer to a particular data item or assembly instruction from another part of the assembly program. Labels will be used extensively when you begin programming, but for now, let’s put this idea on hold.

When you begin writing assembly programs, you will follow a specified format; however, each instruction will use the following structure:

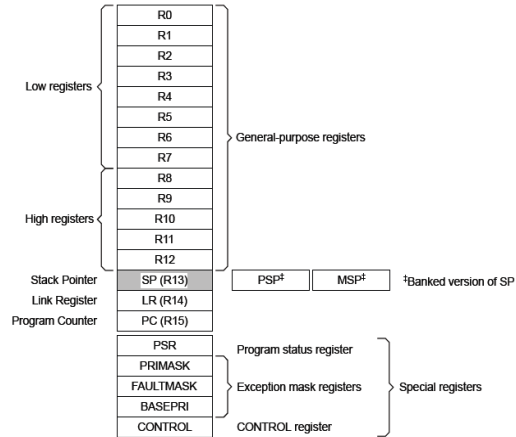
| <u><b>LABEL</b></u> | <u><b>OPERATION</b></u> | <u><b>OPERAND</b></u> | <u><b>COMMENTS</b></u> |
|---------------------|-------------------------|-----------------------|------------------------|
|---------------------|-------------------------|-----------------------|------------------------|

NOTE: The **Operation** field contains the assembly mnemonic for the corresponding machine instruction and cannot start in the first column of the program (only **labels** can exist in the first column). A common practice is to separate each field with a tab (or multiple tabs). The **operand** field contains the arguments required for the machine instruction or assembler directive, with comments following at the end of the instruction.

| <u><b>LABEL</b></u> | <u><b>OPERATION</b></u> | <u><b>OPERAND</b></u> | <u><b>COMMENTS</b></u>  |
|---------------------|-------------------------|-----------------------|-------------------------|
|                     | mov                     | r2, #10               | // move 10d into reg #2 |
|                     | add                     | r4, r2, #11           | // [r2] + 11 → r4       |

Let’s look at the above example and clarify some common nomenclature used in this course. First, notice that the label field is not used in this example, so just ignore it for now. For the first instruction, the Operation field contains the mnemonic for the machine instruction: MOVE. The first operand, r2, is a register name, in this case the name of the destination register that we will move data into. The second operand, #10, is the decimal number 10. The ‘#’ symbol refers to a type of addressing mode, which is discussed below, but in layman’s terms, this instruction is simply telling the CPU to put the value of 10 (base 10) into register r2. The comments field explains the functionality of this instruction using some common nomenclature that will be used in this course. The second instruction instructs the CPU to add the value 11 (base 10) to the contents of register r2 and store the result in register r4. The brackets around the register name in the second comment field indicate the contents of register r2. Note that these comments, describing the behavior of the instructions, are NOT appropriate when you are writing your programs, but they will be used extensively for instructional purposes here and in class.

**Figure 4.1:** The Cortex-M4F register model

**Table 4.1:** Memory Map for TM4C123GXL Development Board

| Address Start | Address End | Memory application                              |
|---------------|-------------|-------------------------------------------------|
| 0x0000.0000   | 0x0003.FFFF | On-Chip Flash ( <i>Where you will program</i> ) |
| 0x0004.0000   | 0x1FFF.FFFF | Reserved                                        |
| 0x2000.0000   | 0x2000.7FFF | Internal SRAM                                   |
| 0x2000.8000   | 0x220F.FFFF | Reserved                                        |
| 0x2210.0000   | 0x3FFF.FFFF | Reserved                                        |

#### 4.7 Running an assembly program using VSCode and Platform.IO:

Start a new project and download the assembly files named *Startup.s* and *PracticeLab.s* from the “Problem4b” folder in the provided zip file and add it to the project’s *src* folder. Make sure to put the provided *platformio.ini* file one directory level higher (as usual).

From now on, all assembly language projects will use this platformio.ini file:

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:lptm4c123gh6pm]
platform = titiva
board = lptm4c123gh6pm
board_build.ldscript = lm4fcpp_blizzard.ld
```

Now build (or assemble) this program as you have done before. Connect the board to the PC and upload the program to the board.

**Figure 1.2:** Program PracticeLab.s. Lines starting with ‘//’ are the comments written in the program.

```
//*****
// Program PracticeLab.s
// Clear memory locations 0x2000.0400 - 0x2000.041F,
// then load these locations with consecutive numbers starting
// with 00.
// 'CONST' is the number of locations operated on.
// 'FIRST' is the address of first memory address.
//*****

//*****
// EQU Directives
// These directives do not allocate memory
//*****
.equ    FIRST, 0x20000400
.equ    CONST, 0x20

//*****
// Program section
//*****
//LABEL DIRECTIVE      VALUE                                COMMENT
.section .text
.syntax unified
.global main
.thumb

main:
    NOP
    LDR    R1,=FIRST      // Initialize registers
    MOV    R0,#0x00      //
    LDR    R2,=CONST      //
loop1:  STRB   R0,[R1]      // Clear memory
        ADD    R1,R1,#1    // Increment address
        SUBS   R2,R2,#1    // Decrement constant
        BNE    loop1
        LDR    R1,=FIRST   // Reset address
        LDR    R2,=CONST   // Reset constant
loop2:  STRB   R0,[R1]      // Store value in memory
        ADD    R0,R0,#1    // Increment value
        ADD    R1,R1,#1    // Increment address
        SUBS   R2,R2,#1    // Decrement constant
        BNE    loop2
done:   WFI                // Infinite loop to
        B      done        // end program
//*****
// End of the program section
//*****
//LABEL      DIRECTIVE      VALUE                                COMMENT
.end
```

When the program is downloaded to the board, it is stored starting at address 0x0000.0000. Enter Debug mode and step through the program. In a *memory view*, examine memory locations 0x0000.026C – 0x0000.0298. These memory locations should contain the object code of the program. We will study instruction encoding later in the semester, but the NOP instruction at the start of main is encoded as the 16-bit machine code instruction 0xBF00. You should be able to find this instruction starting at address 0x0000.026C.

Now look at memory locations 0x2000.0400 – 0x2000.041F. This program modifies these memory locations by first loading 0x00 into all of them, then storing consecutive numbers starting with 0x00. Step through the program by pressing F11. Watch as the program clears the memory locations and then populates it with consecutive numbers. Also pay attention to the values in the registers and how they relate to the instructions being executed. **It is critical that you understand how to step through your program in debug mode and be able to examine registers and memory locations.**

Call the lab TA and demonstrate the program working. Step through the program in debug mode. Walk through the code and describe generally what the code is doing.

#### 4.8 Prework:

Referring back to Section 4.5, Procedures 1 and 2, create the structure called course and typedef this structure as Course and show to your TA and get his sign-off before you begin work on this lab. These creations can be handwritten for this demo.

#### 4.9 Lab report:

The lab report is due (11:59PM) of the week after the conclusion of this lab. Monday labs are due on Monday, Wednesday labs are due on Wednesday, and Friday labs are due on Friday. For the lab write up, include the following:

1. A copy of your working .c/.h/.s files for the program. (All .c/.h/.s files) – 40 pts
2. A brief discussion of the objectives of the lab and the procedures performed in the lab. Includes checkoffs (PDF) – 40 pts
3. Answers to any **questions** in the discussion, procedure, or question sections of the lab. (PDF) – 20 pts

***Submit your files separately to Canvas, do not zip them together. Please keep in mind that you must have your work checked off (look for the green sections) by the TA to receive credit for the lab.***

## Appendix A: ASCII Table

Decimal - Binary - Octal - Hex – ASCII  
Conversion Chart

| Decimal | Binary   | Octal | Hex | ASCII | Decimal | Binary   | Octal | Hex | ASCII | Decimal | Binary   | Octal | Hex | ASCII | Decimal | Binary   | Octal | Hex | ASCII |
|---------|----------|-------|-----|-------|---------|----------|-------|-----|-------|---------|----------|-------|-----|-------|---------|----------|-------|-----|-------|
| 0       | 00000000 | 000   | 00  | NUL   | 32      | 00100000 | 040   | 20  | SP    | 64      | 01000000 | 100   | 40  | @     | 96      | 01100000 | 140   | 60  | `     |
| 1       | 00000001 | 001   | 01  | SOH   | 33      | 00100001 | 041   | 21  | !     | 65      | 01000001 | 101   | 41  | A     | 97      | 01100001 | 141   | 61  | a     |
| 2       | 00000010 | 002   | 02  | STX   | 34      | 00100010 | 042   | 22  | "     | 66      | 01000010 | 102   | 42  | B     | 98      | 01100010 | 142   | 62  | b     |
| 3       | 00000011 | 003   | 03  | ETX   | 35      | 00100011 | 043   | 23  | #     | 67      | 01000011 | 103   | 43  | C     | 99      | 01100011 | 143   | 63  | c     |
| 4       | 00000100 | 004   | 04  | EOT   | 36      | 00100100 | 044   | 24  | \$    | 68      | 01000100 | 104   | 44  | D     | 100     | 01100100 | 144   | 64  | d     |
| 5       | 00000101 | 005   | 05  | ENQ   | 37      | 00100101 | 045   | 25  | %     | 69      | 01000101 | 105   | 45  | E     | 101     | 01100101 | 145   | 65  | e     |
| 6       | 00000110 | 006   | 06  | ACK   | 38      | 00100110 | 046   | 26  | &     | 70      | 01000110 | 106   | 46  | F     | 102     | 01100110 | 146   | 66  | f     |
| 7       | 00000111 | 007   | 07  | BEL   | 39      | 00100111 | 047   | 27  | '     | 71      | 01000111 | 107   | 47  | G     | 103     | 01100111 | 147   | 67  | g     |
| 8       | 00001000 | 010   | 08  | BS    | 40      | 00101000 | 050   | 28  | (     | 72      | 01001000 | 110   | 48  | H     | 104     | 01101000 | 150   | 68  | h     |
| 9       | 00001001 | 011   | 09  | HT    | 41      | 00101001 | 051   | 29  | )     | 73      | 01001001 | 111   | 49  | I     | 105     | 01101001 | 151   | 69  | i     |
| 10      | 00001010 | 012   | 0A  | LF    | 42      | 00101010 | 052   | 2A  | *     | 74      | 01001010 | 112   | 4A  | J     | 106     | 01101010 | 152   | 6A  | j     |
| 11      | 00001011 | 013   | 0B  | VT    | 43      | 00101011 | 053   | 2B  | +     | 75      | 01001011 | 113   | 4B  | K     | 107     | 01101011 | 153   | 6B  | k     |
| 12      | 00001100 | 014   | 0C  | FF    | 44      | 00101100 | 054   | 2C  | ,     | 76      | 01001100 | 114   | 4C  | L     | 108     | 01101100 | 154   | 6C  | l     |
| 13      | 00001101 | 015   | 0D  | CR    | 45      | 00101101 | 055   | 2D  | -     | 77      | 01001101 | 115   | 4D  | M     | 109     | 01101101 | 155   | 6D  | m     |
| 14      | 00001110 | 016   | 0E  | SO    | 46      | 00101110 | 056   | 2E  | .     | 78      | 01001110 | 116   | 4E  | N     | 110     | 01101110 | 156   | 6E  | n     |
| 15      | 00001111 | 017   | 0F  | SI    | 47      | 00101111 | 057   | 2F  | /     | 79      | 01001111 | 117   | 4F  | O     | 111     | 01101111 | 157   | 6F  | o     |
| 16      | 00010000 | 020   | 10  | DLE   | 48      | 00110000 | 060   | 30  | 0     | 80      | 01010000 | 120   | 50  | P     | 112     | 01110000 | 160   | 70  | p     |
| 17      | 00010001 | 021   | 11  | DC1   | 49      | 00110001 | 061   | 31  | 1     | 81      | 01010001 | 121   | 51  | Q     | 113     | 01110001 | 161   | 71  | q     |
| 18      | 00010010 | 022   | 12  | DC2   | 50      | 00110010 | 062   | 32  | 2     | 82      | 01010010 | 122   | 52  | R     | 114     | 01110010 | 162   | 72  | r     |
| 19      | 00010011 | 023   | 13  | DC3   | 51      | 00110011 | 063   | 33  | 3     | 83      | 01010011 | 123   | 53  | S     | 115     | 01110011 | 163   | 73  | s     |
| 20      | 00010100 | 024   | 14  | DC4   | 52      | 00110100 | 064   | 34  | 4     | 84      | 01010100 | 124   | 54  | T     | 116     | 01110100 | 164   | 74  | t     |
| 21      | 00010101 | 025   | 15  | NAK   | 53      | 00110101 | 065   | 35  | 5     | 85      | 01010101 | 125   | 55  | U     | 117     | 01110101 | 165   | 75  | u     |
| 22      | 00010110 | 026   | 16  | SYN   | 54      | 00110110 | 066   | 36  | 6     | 86      | 01010110 | 126   | 56  | V     | 118     | 01110110 | 166   | 76  | v     |
| 23      | 00010111 | 027   | 17  | ETB   | 55      | 00110111 | 067   | 37  | 7     | 87      | 01010111 | 127   | 57  | W     | 119     | 01110111 | 167   | 77  | w     |
| 24      | 00011000 | 030   | 18  | CAN   | 56      | 00111000 | 070   | 38  | 8     | 88      | 01011000 | 130   | 58  | X     | 120     | 01111000 | 170   | 78  | x     |
| 25      | 00011001 | 031   | 19  | EM    | 57      | 00111001 | 071   | 39  | 9     | 89      | 01011001 | 131   | 59  | Y     | 121     | 01111001 | 171   | 79  | y     |
| 26      | 00011010 | 032   | 1A  | SUB   | 58      | 00111010 | 072   | 3A  | :     | 90      | 01011010 | 132   | 5A  | Z     | 122     | 01111010 | 172   | 7A  | z     |
| 27      | 00011011 | 033   | 1B  | ESC   | 59      | 00111011 | 073   | 3B  | ;     | 91      | 01011011 | 133   | 5B  | [     | 123     | 01111011 | 173   | 7B  | {     |
| 28      | 00011100 | 034   | 1C  | FS    | 60      | 00111100 | 074   | 3C  | <     | 92      | 01011100 | 134   | 5C  | \     | 124     | 01111100 | 174   | 7C  |       |
| 29      | 00011101 | 035   | 1D  | GS    | 61      | 00111101 | 075   | 3D  | =     | 93      | 01011101 | 135   | 5D  | ]     | 125     | 01111101 | 175   | 7D  | }     |
| 30      | 00011110 | 036   | 1E  | RS    | 62      | 00111110 | 076   | 3E  | >     | 94      | 01011110 | 136   | 5E  | ^     | 126     | 01111110 | 176   | 7E  | ~     |
| 31      | 00011111 | 037   | 1F  | US    | 63      | 00111111 | 077   | 3F  | ?     | 95      | 01011111 | 137   | 5F  | _     | 127     | 01111111 | 177   | 7F  | DEL   |